

Discovering Colocation Patterns from Spatial Data Sets: A General Approach

Dokumentacja końcowa

05.05.2026

Mateusz Krajewski

1 Dane

Model danych



Dane wejściowe są reprezentowane jako tabela zdarzeń przestrzennych. Każdy wiersz opisuje jedną instancję obiektu przestrzennego:

Kolumna	Znaczenie
<code>instance_id</code>	unikalny identyfikator instancji
<code>feature_type</code>	typ cechy przestrzennej, np. <code>amenity=restaurant</code>
<code>x</code>	współrzędna X w metrycznym układzie odniesienia
<code>y</code>	współrzędna Y w metrycznym układzie odniesienia

Dane wykorzystywane w projekcie pochodzą z *OpenStreetMap* i dotyczą Warszawy oraz Pragi.

Skrypt `src/dataset.py` pobiera obiekty OSM, przypisuje im jedną etykietę `feature_type`, przekształca geometrię do projekcji metrycznej i zapisuje punkty reprezentatywne obiektów do pliku `.csv`.

Przykład pobrania danych:

```
uv run dataset Warsaw Poland
uv run dataset Prague Czechia
```

Przykładowy plik danych



data\warsaw_osm_events.csv:

```
instance_id,feature_type,x,y
0,amenity=restaurant,496304.0553927579,5780716.109113953
1,railway=tram_stop,498361.39182775735,5784456.196142845
2,railway=tram_stop,498767.43019384245,5784370.355951676
3,railway=tram_stop,500762.04196613963,5786585.82556599
4,highway=bus_stop,492003.8327595345,5782678.6434959
5,railway=tram_stop,496888.13229768444,5781738.21188593
6,railway=tram_stop,499914.4900040053,5785392.75686958
7,railway=tram_stop,501100.0635658219,5785826.252234489
8,railway=tram_stop,500818.0169793588,5786559.896314677
9,railway=tram_stop,500439.43444095616,5787573.907411887
10,railway=tram_stop,500195.8076818618,5788080.132238101
```

Relacja sąsiedztwa



W artykule relacja sąsiedztwa R jest parametrem algorytmu i może być dowolna, o ile odpowiada semantyce dziedziny. W tej implementacji przyjęto relację euklidesową:

Relacja sąsiedztwa R

$$(i, j) \in R \Leftrightarrow \text{dist}_E(i, j) \leq d$$

Do wyznaczania sąsiedztwa używany jest `scipy.spatial.cKDTree.query_pairs`.

W strukturze sąsiedztwa przechowywane są tylko pary różnych instancji.

2 Relacje

Table instance i row instance



Dla kolokacji $C = \{f_1, f_2, \dots, f_k\}$:

- **row instance** to krotka instancji $(i_1, i_2, \dots, i_k)$, gdzie każda instancja ma odpowiedni typ cechy i wszystkie instancje w krotce są wzajemnie sąsiednie;
- **table instance** to zbiór wszystkich takich krotek dla danej kolokacji.

Przykład dla kolokacji (A, B) :

```
table_instance((A, B)) = [(A.1, B.1), (A.2, B.4), (A.3, B.4)]
```

Participation ratio i participation index



Dla kolokacji C i cechy f_i :

Participation ratio

$PR(C, f_i) = \text{liczba różnych instancji } f_i \text{ występujących w } table_instance(C) / \text{liczba wszystkich instancji } f_i \text{ w danych}$

Participation index

$PI(C) = \min PR(C, f_i) \text{ dla wszystkich } f_i \text{ należących do } C$

Reguły kolokacji



Dla prevalent colocation $C = C1 \text{ union } C2$ generowana jest reguła: $C1 \Rightarrow C2$

Warunkiem przyjęcia reguły jest: $\text{conditional_probability}(C1 \Rightarrow C2) \geq \text{min_cp}$

W implementacji:

Reguła kolokacji

$$\text{cp}(C1 \Rightarrow C2) = \frac{\text{liczba różnych projekcji } C1 \text{ z tabeli } \text{table_instance}(C1 \text{ union } C2)}{\text{liczba row instances w tabeli } \text{table_instance}(C1)}$$

Dla poprzednika jednoelementowego mianownikiem jest liczba wszystkich instancji tej cechy.

3 Algorytm

Wejście:

- E - zbiór instancji zdarzeń: (instance_id, feature_type, x, y)
- d - próg sąsiedztwa przestrzennego
- min_pi - minimalny participation index
- min_cp - minimalne conditional probability

Wyjście:

- prevalent - zbiór prevalent colocations z wartościami PI
- rules - zbiór reguł kolokacji z PI i cp

Algorytm:

1. Wczytaj współrzędne i typy cech.
2. Policz liczbę instancji dla każdego `feature_type`.
3. Utwórz `P1`:
 - dla każdego `feature_type f`:
 - `P1` zawiera kolokację (`f`)
 - `table_instance((f))` = wszystkie instancje `f`
 - `PI((f))` = 1
4. Zbuduj relację sąsiedztwa `R`:
 - `pairs` = `KDTree.query_pairs(distance=d)`
 - `adjacency` = lista sąsiadów dla każdej instancji
5. Wygeneruj kandydatów rozmiaru 2:

```
dla każdej pary sąsiadów (i, j) z pairs:  
    jeżeli feature(i) != feature(j):  
        dodaj uporządkowaną krotkę (i, j)  
        do table_instance((feature_i, feature_j))
```

6. Dla każdej kolokacji rozmiaru 2:

```
    oblicz PR i PI  
    jeżeli PI >= min_pi:  
        zapisz jako prevalent
```

7. Ustaw $k = 2$.

Dopóki istnieją prevalent colocations rozmiaru k :

```
     $C_{\{k+1\}} = \text{apriori\_gen}(P_k)$ 
```

Pseudokod (iv)



```
dla każdego kandydata c w C_{k+1}:  
    p = c bez ostatniej cechy  
    q = c bez przedostatniej cechy  
    połącz table_instance(p) i table_instance(q)  
    po wspólnym prefiksie instancji  
    sprawdź warunek sąsiedztwa dla dwóch nowych instancji  
  
    jeżeli powstała niepusta table_instance(c):  
        oblicz PR i PI  
        jeżeli PI >= min_pi:  
            zapisz c jako prevalent  
  
k = k + 1
```

8. Dla każdej prevalent colocation c :
 - dla każdego niepustego właściwego podzbioru antecedent:
 - consequent = $c \setminus \text{antecedent}$
 - $cp = \frac{\text{liczba różnych projekcji antecedent z table_instance}(c)}{\text{liczba instancji antecedent}}$
 - jeżeli $cp \geq \text{min_cp}$:
 - dodaj regułę $\text{antecedent} \Rightarrow \text{consequent}$
9. Zwróć prevalent colocations i rules.

Generowanie kandydatów Apriori



Funkcja `apriori_gen` działa zgodnie z klasycznym krokiem Apriori:

1. Łączy kolokacje rozmiaru k , które mają wspólny prefiks.
2. Tworzy kandydata rozmiaru $k + 1$.
3. Usuwa kandydata, jeżeli dowolny jego podzbiór rozmiaru k nie był prevalent.

4 Wyniki testów

`test_apriori_gen.py`

generowanie kandydatów i przycinanie Apriori

`test_build_neighbors.py`

budowa relacji sąsiedztwa, w tym przypadki brzegowe

`test_generate_size2_geometric.py`

generowanie `table instances` dla par

`test_join_combinatorial.py`

join kombinatoryczny dla kolokacji większych niż 2

`test_participation_index.py`

obliczanie `participation ratio` i `participation index`

`test_generate_rules.py`

generowanie reguł i filtrowanie po `min_cp`

`test_smoketest.py`

test regresyjny odtwarzający wartości PI z przykładu z artykułu

Zbiór	Liczba zdarzeń	Najliczniejsze typy cech
Warszawa	13235	highway=bus_stop, amenity=restaurant, shop=convenience, amenity=atm, amenity=cafe
Praga	11695	highway=bus_stop, amenity=restaurant, amenity=cafe, shop=convenience, leisure=park

Wyniki liczbowe



Wyniki liczbowe odpowiadające parametrom:

- distance 200
- prevalence 0.4
- conditional 0.8

Zbiór	Prevalent colocations	Reguły
Warszawa	19	12
Praga	48	58

Najsilniejsze kolokacje dla Warszawy



Kolokacja	Rozmiar	PI	Liczba row instances
amenity=restaurant, shop=convenience	2	0.670	4379
amenity=atm, amenity=pharmacy	2	0.665	1158
amenity=atm, amenity=restaurant	2	0.641	3945
amenity=cafe, amenity=restaurant	2	0.620	5162
amenity=atm, amenity=cafe	2	0.609	2560

Najsilniejsze kolokacje dla Pragi



Kolokacja	Rozmiar	PI	Liczba row instances
amenity=restaurant, shop=convenience	2	0.755	6860
amenity=cafe, amenity=restaurant	2	0.738	15004
amenity=atm, amenity=pharmacy	2	0.699	870
amenity=atm, amenity=cafe	2	0.699	3724
amenity=cafe, shop=convenience	2	0.659	3163

Najsilniejsze reguły dla Warszawy



Reguła	Rozmiar	PI	cp
amenity=bank => amenity=atm	2	0.555	0.927
amenity=atm, amenity=cafe => amenity=restaurant	3	0.457	0.917
amenity=atm, amenity=restaurant => shop=convenience	3	0.415	0.866
amenity=cafe => amenity=restaurant	2	0.620	0.846
amenity=restaurant => highway=bus_stop	2	0.456	0.844

Najsilniejsze reguły dla Pragi



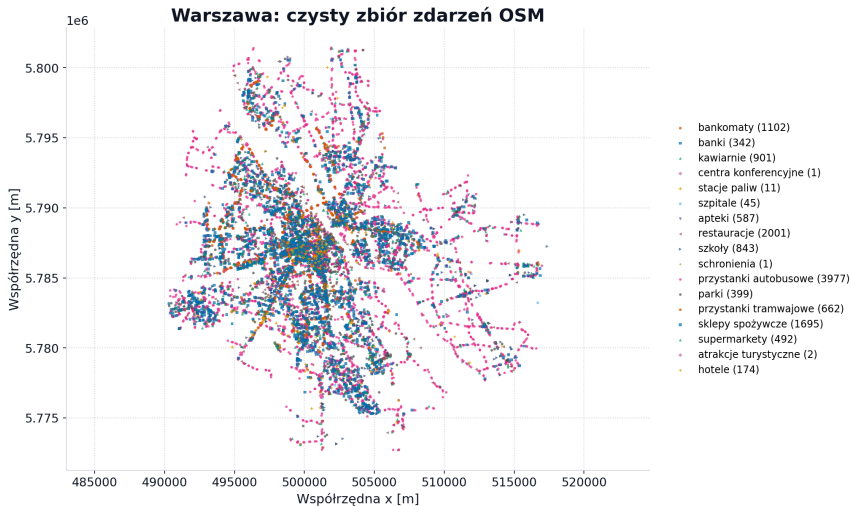
Reguła	Rozmiar	PI	cp
amenity=atm, amenity=cafe, tourism=hotel => amenity=restaurant	4	0.432	0.999
amenity=atm, amenity=cafe, shop=convenience => amenity=restaurant	4	0.404	0.997
amenity=cafe, tourism=hotel => amenity=restaurant	3	0.517	0.996
amenity=atm, tourism=hotel => amenity=restaurant	3	0.455	0.994
shop=convenience, tourism=hotel => amenity=restaurant	3	0.412	0.991

5 Wykresy

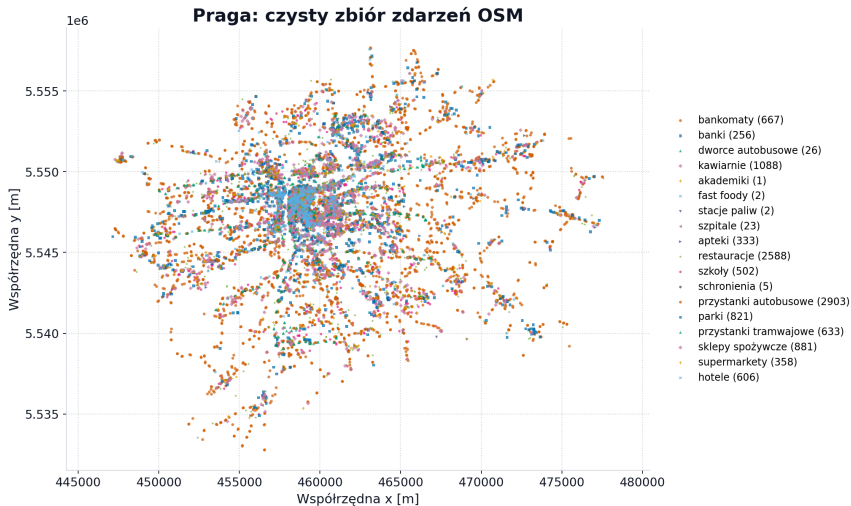
Projekt generuje dwa typy wykresów dla każdego zbioru, które zawierają:

- liczbę prevalent collocations według rozmiaru,
- ranking największych wartości PI,
- rozrzut reguł względem cp i PI,
- ranking reguł według cp,
- czystą wizualizację zdarzeń wejściowych bez nałożonych wyników algorytmu,
- przestrzenne nałożenie instancji uczestniczących w wybranych kolokacjach. Szare punkty stanowią tło całego zbioru, a kolorowe markery wskazują instancje, które faktycznie uczestniczą w table instances danej kolokacji.

Czysty dataset: Warszawa



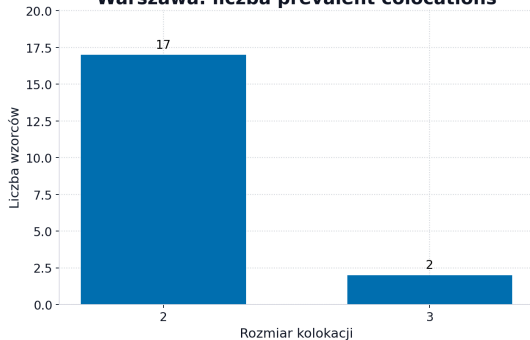
Czysty dataset: Praga



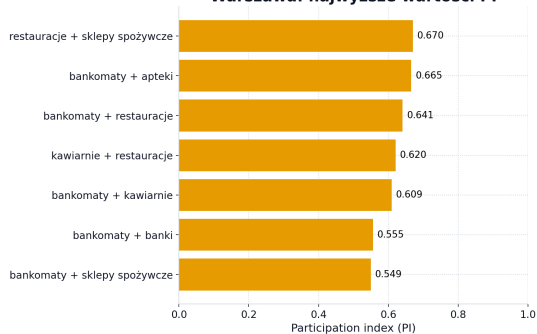
Wyniki: Warszawa



Warszawa: liczba prevalent colocations



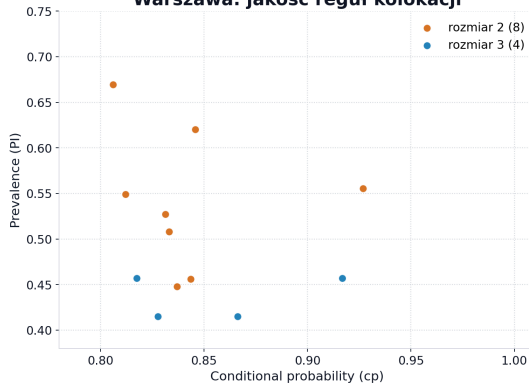
Warszawa: najwyższe wartości PI



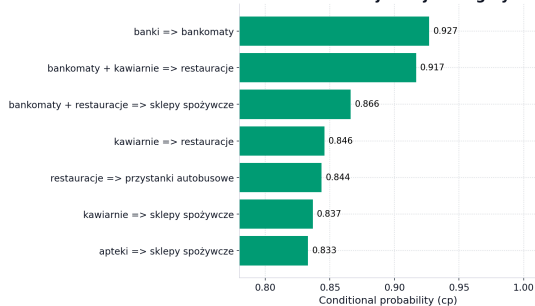
Reguły: Warszawa



Warszawa: jakość reguł kolokacji



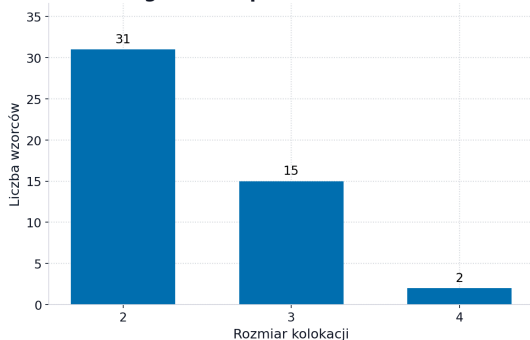
Warszawa: najsilniejsze reguły



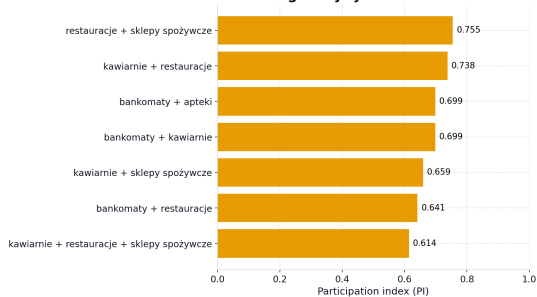
Wyniki: Praga



Praga: liczba prevalent colocations



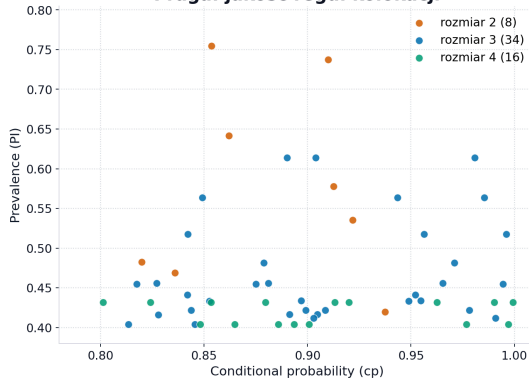
Praga: najwyższe wartości PI



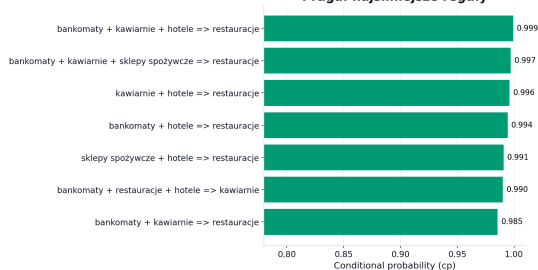
Reguły: Praga



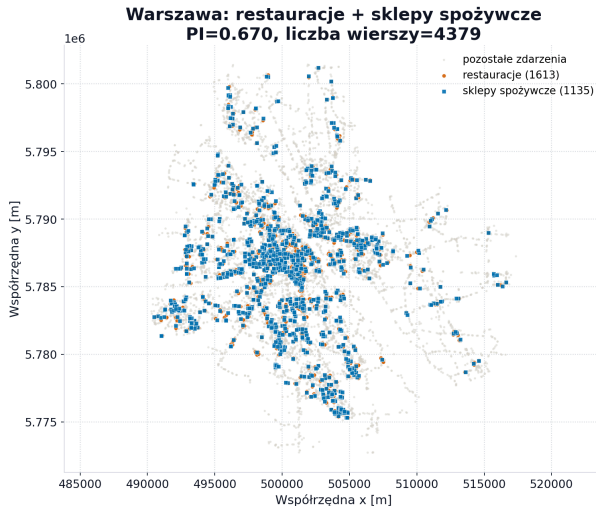
Praga: jakość reguł kolokacji



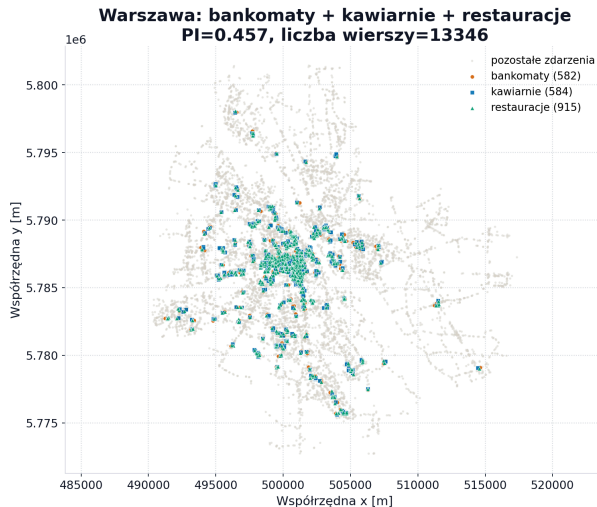
Praga: najsilniejsze reguły



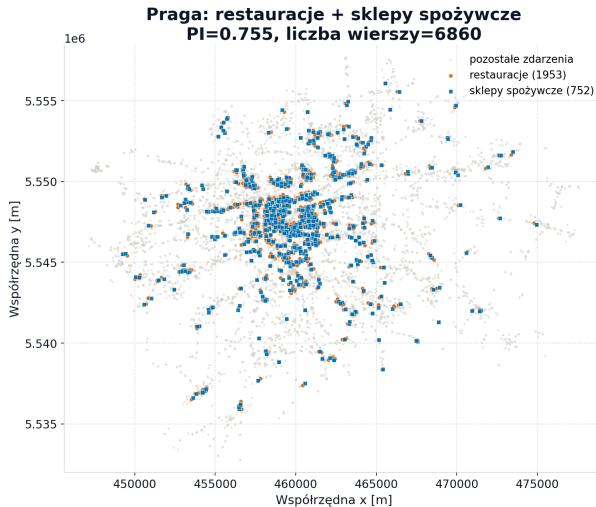
Spatial colocation: Warszawa, rozmiar 2



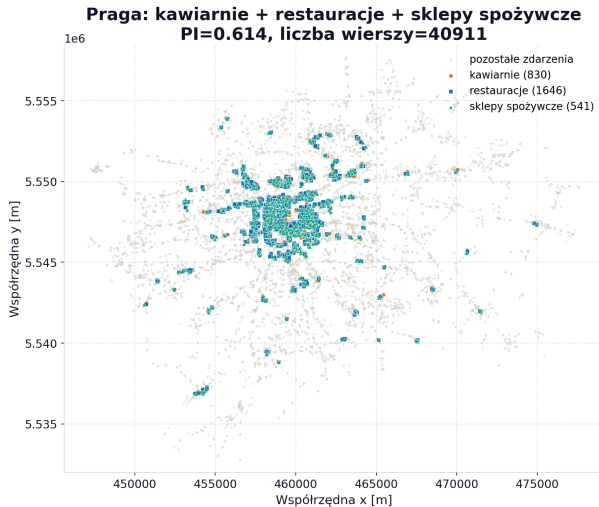
Spatial colocation: Warszawa, rozmiar 3



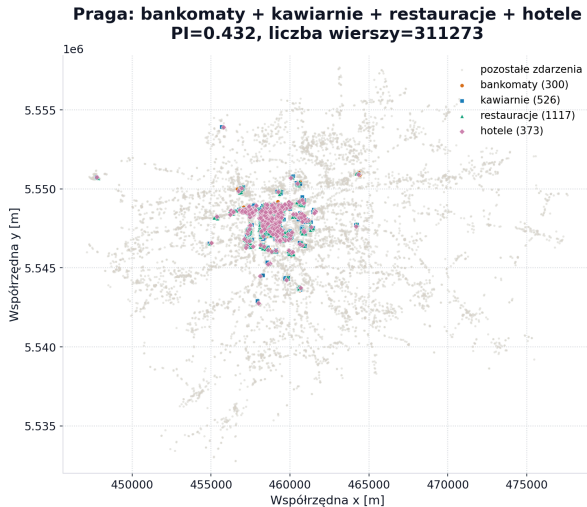
Spatial colocation: Praga, rozmiar 2



Spatial colocation: Praga, rozmiar 3



Spatial colocation: Praga, rozmiar 4



- Implementacja zachowuje główną logikę algorytmu z artykułu.
- W obu miastach bardzo wysokie PI osiągają kolokacje łączące restauracje, sklepy convenience, kawiarnie, bankomaty i apteki. Jest to zgodne z intuicją dla centrów miejskich, gdzie usługi konsumenckie występują blisko siebie.
- Praga ma więcej wykrytych kolokacji i reguł niż Warszawa przy tych samych progach. W wynikach Pragi część bardzo silnych reguł zawiera `tourism=hotel`, co sugeruje koncentrację hoteli w obszarach z restauracjami i kawiarniami.
- Reguły o wysokim `cp` nie zawsze oznaczają najwyższe PI. `cp` mierzy warunkową przewidywalność konsekwentu z poprzednika, a PI dodatkowo wymaga udziału wszystkich cech kolokacji.

Dziękuję za uwagę